Sparsh Bhimrajka s3598853
Jay Bakshi s3715043
Aleksandar Nikolov s3564681
Martin Goranov s3595412

Assignment 3

Security analysis report

Introduction

The report shows the security principles we used to make our application safe and protected from malicious events. These principles are listed below:

Security

Password security

Our system uses Argon2id hashing with a secret salting schema to protect the customer's password. So this protects the user's password even if someone steals the database. So the idea behind that is that we check and track users' passwords not by the plain schema, but against stored hashes.

SQL Injection protection

We protected our database queries using PreparedStatements. This prevents attackers from putting malicious SQL code through the user input. So when a ticket is created, it is safely stored into the database.

Role based access control

The system checks user roles (Support, Consultant, Customer) before allowing actions. A customer cannot update a ticket's status and a

consultant cannot assign tickets. So this checks if an unauthorized user is trying to access kinds of features.

Audit logging
Every important action (login, ticket creation, updates) is logged inside with the user, timestamp and details, which are important credentials. This allows us to track the things that are done, and helps us to understand if a malicious action is done.

Testing

User Testing
We conducted user testing using all of the 3 roles to make sure that from a real user's perspective (customer), the system behaves as expected and is simple enough to navigate throughout the website. Test users were all teammates, and everyone performed common tasks such as creating tickets, assigning tickets and updating ticket statuses using different roles each. This helped us identify if there are any usability issues and also helped to verify that role based access was functioning properly and correctly.

Unit Testing
We created unit tests for individual components and backend functions to make sure that independently, they work correctly each. For example, we tested authentication, ticket creation correctness and role validation. This helped test that the small changes didn’t accidentally break or stop the functioning ones from working.

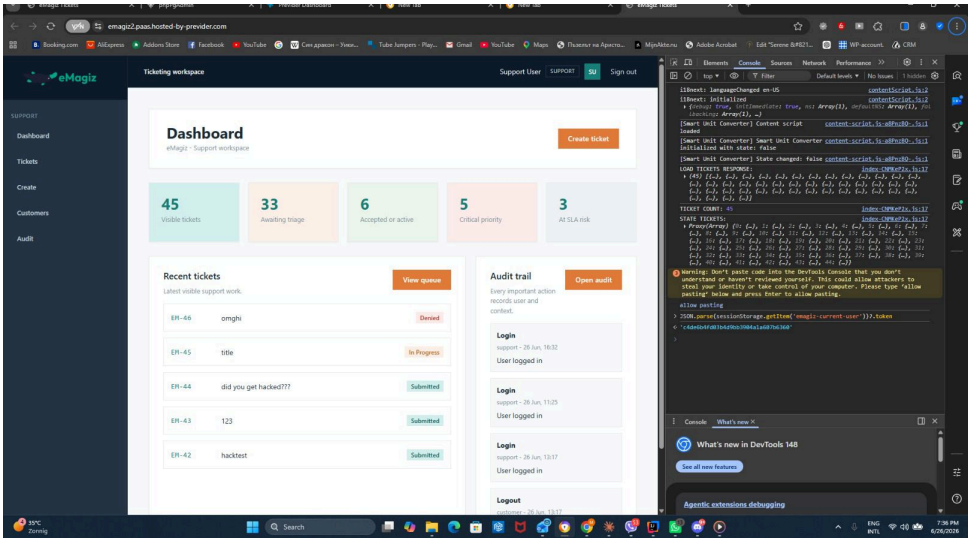
Integration Testing
We performed integration tests to make sure that different parts of the system work together like cogwork. This included testing both, the connection and communication between the frontend, backend API and database. For example, creating a ticket through the frontend was tested and checked if it was correctly being processed by the backend and securely being stored in our database.

User story testing

All of our ‘must have’ and ‘should have’ requirements from the MoSCoW model in assignment 1 have been implemented and tested for proper functionality in addition to all the other tests which have been done by us. Testing included verifying that each feature met its acceptance criteria. We were not able to implement the ‘could have’ user stories due to a time and resources limitation. These features can be worked upon in the future.

We have also tested more security scenarios, namely,(Session/cookie hijacking, Accessing protected routes without authentication, Accessing another user's data directly, Invalid JWT/token usage (if using tokens), SQL injection attempts, and Input validation). The results we achieved show that we have been able to make our system as secure as necessary and fix our previous vulnerabilities.

(image of session/cookie hijacking attempt)



Deployment Security

HTTPS

The deployed website uses HTTP‘S’ to encrypt all communication between the browser and server. This helps to prevent attackers from intercepting any sensitive information such as login details. Also, the

system uses secure session tokens to retain authentication after logging in. This prevents credentials from being repeatedly written by a user and ensures only authenticated users can use the website, and makes it less vulnerable to hackers or identity thefts.

Secure Configuration
Database credentials, password security, and other sensitive information has been moved out from the source code and to the database- stored as environment variables, making credential rotation much easier.

Automated Testing
Automated tests are now conducted to make sure that the website continues to work after repeated incremental changes are made, helping to maintain system stability.

CI/CD Verification
Integration tests run as part of the GitLab pipeline. Every push is automatically verified before deployment, making sure that the database amends, API endpoints and the workflow continue to function correctly. Each commit is monitored by all teammates equally, to make sure that it's by a verified trusted source.

Security Testing Results and Implications

Method of testing

We attempted black-box testing against our eMagiz ticketing system. We used the application in the normal customer mode with that we used developer tools to inspect the network traffic that was generated by the frontend.

Our objective was to test whether our authentication/authorisation systems were correctly enforced by our backend.

API requests were monitored through the network tab. Significant focus was on how the application identified the currently logged-in.

Results

The analysis revealed that API requests contained a userId parameter, which appeared to identify the user making the request.

Example:
GET /api/tickets?userId=3
A userid parameter was being used to identify the users. By changing this value, accessing other users' data was possible. This revealed that authorisation checks were not being performed correctly by the backend.

Implications

The vulnerability allows a user without access to attempt and succeed in getting the resources and information that belong to other users.

Potential consequences include:
Unauthorized access to tickets. It can also include leak of sensitive customer data and information, it can expose internal support data and change the role based permissions or rather abuse them. This vulnerability is considered with really high severity because it affects the confidentiality and the authorization of users.

Recommendation

We have to make the backend never trust user identifiers supplied by the client.
Instead we should implement server side authentications and better session-based authentication or JWT authentication.

Conclusion

The black box testing process successfully identified a vulnerability through broken access control. Even if the application provided authentication functionality, authorization decisions can be decided by the user id supplied in the requests. Also the server-side validation is

really important and required to ensure that users can only access resources that they are authorized to view.